

Backend Flow — Explained Simply

ChatWave · Real-Time Chat Application

How a request travels through your server —

Express routes → middleware → controllers → Mongoose models → MongoDB,
plus real-time messaging with Socket.IO.

Backend: Node.js + Express + MongoDB (Mongoose) + JWT + Socket.IO

1. The big picture

Your backend is an **Express** server that listens for requests from the React app, talks to a **MongoDB** database through **Mongoose**, and pushes live messages using **Socket.IO**.

Every normal request follows this path:

Client	Middleware	Route	Controller	Model	MongoDB
--------	------------	-------	------------	-------	---------

And the response travels back the same way as JSON. For live chat, the server also opens a **websocket** connection (Socket.IO) that stays open to push new messages instantly.

The folders (layers)

Folder / File	Responsibility
index.js	Starts the server, sets middleware, mounts routes
routes/	Maps a URL + method to a controller function
middleware/	Runs before controllers (e.g. auth check)
controllers/	The actual logic for each request
models/	Mongoose schemas — the shape of your data
config/	Database connection
socket/	Socket.IO real-time setup

2. A request's journey (step by step)

- 1. Request arrives** — the React app calls e.g. POST /api/v1/message/send/:id (with the auth cookie).
- 2. Global middleware** runs: CORS (allow the frontend), express.json() (parse the body), cookieParser() (read cookies).
- 3. Router** matches the URL to a controller — here, messageRoute sends it to sendMessage.
- 4. Auth middleware** (isAuthenticated) reads the JWT cookie, verifies it, and sets req.id = your userId.
- 5. Controller** runs the logic (create the message, save the conversation).
- 6. Model (Mongoose)** reads/writes the data in MongoDB.
- 7. Response** — the controller sends JSON back; Socket.IO may also push a live event.

3. Authentication flow (JWT + cookies)

Register → POST /api/v1/user/register

- Validates fields and that password == confirmPassword.
- Hashes the password with **bcrypt** (never stored in plain text).
- Creates the user with a generated avatar; returns success.

Login → POST /api/v1/user/login

- Finds the user, compares the password with **bcrypt.compare**.
- On success, signs a **JWT** token and stores it in an **httpOnly** cookie.
- Returns your profile (id, name, username, photo, gender, status, joined).

```
jwt.sign({userId}, SECRET, {expiresIn:"1d"})  
  .cookie("token", token, {httpOnly:true, sameSite:"strict"})
```

Protect routes → isAuthenticated middleware

For any protected route, this middleware reads the cookie, verifies the token, and attaches your id:

```
const token = req.cookies.token;  
const decode = jwt.verify(token, SECRET);  
req.id = decode.userId; // now controllers know who you are  
next();
```

Logout → GET /api/v1/user/logout

Clears the cookie by setting it empty with maxAge 0.

4. All API endpoints

#	Method	Endpoint	Auth	Controller
1	POST	/api/v1/user/register	No	register
2	POST	/api/v1/user/login	No	login
3	GET	/api/v1/user/logout	No	logout
4	POST	/api/v1/user/profile/update	Yes	updateProfile
5	GET	/api/v1/user/	Yes	getOtherUsers
6	POST	/api/v1/message/send/:id	Yes	sendMessage
7	GET	/api/v1/message/:id	Yes	getMessage

5. Data models & how they relate

User

fullName, username (unique), password (hashed), profilePhoto, gender, showStatus, timestamps

Message

senderId → User, receiverId → User, message, timestamps

Conversation

participants: [User, User], messages: [Message], timestamps

Relationship: A **Conversation** links two users and holds a list of **Message** IDs. Each Message stores who sent it and to whom. This lets the app load a whole chat by its conversation.

User A	Conversation (A + B)	Message list	User B
--------	----------------------	--------------	--------

6. Sending & loading messages

Send → sendMessage

1. Find the Conversation between sender & receiver; if none exists, **create** it.
2. **Create** the Message document (senderId, receiverId, text).
3. Push the message's id into the conversation, then **save** both.
4. If the receiver is online, **Socket.IO** pushes 'newMessage' to them instantly.
5. Respond to the sender with the new message.

Load → getMessage

Finds the conversation between the two users and **.populate('messages')** to return the full message objects (not just their ids).

7. Real-time with Socket.IO

Socket.IO keeps a live connection open so messages and presence update without refreshing.

- When a user connects, the server saves **userId** → **socketId** in a map (userSocketMap).
- It broadcasts **getOnlineUsers** (the list of online user ids) to everyone.
- To deliver a message, **getReceiverSocketId(receiverId)** finds that user's socket, and the server emits **newMessage** only to them.
- On disconnect, the user is removed from the map and the online list is broadcast again.

```
io.on("connection", (socket) => {  
  const userId = socket.handshake.query.userId;  
  userSocketMap[userId] = socket.id;  
  io.emit("getOnlineUsers", Object.keys(userSocketMap));  
});
```

8. End-to-end: one message, start to finish

Login (cookie)	GET users	GET messages	POST send	socket newMessage
----------------	-----------	--------------	-----------	-------------------

1. You log in → server sets the JWT cookie.
2. App calls GET /user → getOtherUsers returns your friends.
3. You open a chat → GET /message/:id → getMessage returns the history.
4. You send → POST /message/send/:id → sendMessage saves it to MongoDB.
5. The server emits 'newMessage' over Socket.IO → your friend sees it live.

In one line: a request passes through middleware, hits a route, runs a controller, which uses a Mongoose model to read/write MongoDB and returns JSON — while Socket.IO pushes live updates on the side.